Built on Drunix (NPCI's Hyperledger Fabric 2.5 fork) · Human-in-the-loop · Evaluated in simulation

QueueShield × Drunix

Stop UPI status-check storms before they lock in: find the sick bank, let a human approve bounded actions, and resolve unknown payments from a shared ledger, not by polling the switch.

UPI, Sep 2026
24.07 bn
payments in one month, avg **802 M/day** (record)

2025 outages
3
26 Mar · 2 Apr · 12 Apr: NPCI blamed status-check traffic

Retry storm (S4) SIM
0 / 5
runs locked in with QueueShield, vs **3/5** no control, **2/5** NPCI rule

Apr-2025 replay (S6) SIM
−61%
switch checks per unknown vs no control (7.2 vs 18.4)

THE PITCH, IN THE FAQ'S RECOMMENDED FLOW

1 · Problem — Outages, the storm loop, the gaps

2 · User journey — One payment, five actors

3 · Solution — Detect → forecast → recommend → resolve

4 · Architecture — 5 modules, 17 JSON contracts

5 · Drunix flow — Attest once, push to all

6 · Prototype — Console, replays, arena

7 · Impact — Results, honest scorecard, plan

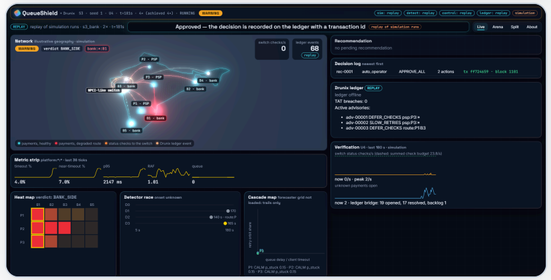
Localise before you act. Name the PSP, bank, route or switch, not "errors are up"; never fail over across a shared bank.

Push, don't poll. The bank that knows the outcome attests it once on Drunix; every PSP is pushed the answer.

Machines recommend, humans decide. Every approval and rejection is recorded on the ledger with a tx id.

BUILT — M1 Go chaincode + TS gateway · M2 Python simulator · M3 Python detection · M4 Python control · M5 React + three.js console · 17 JSON Schemas · 106 automated tests · ≈11.7k lines of code





Bank slowdown: localiser names bank B1, recommendation waits for approval



April-2025 replay: 2,570 switch checks/s with no control vs 38 with QueueShield SIM

Read this first. Every outcome number in this deck comes from our own simulator and is labelled SIMULATION. All data is synthetic. The ledger demo runs on the gateway's mock mode (same API and events). This is a scaled analogue of NPCI's April 2025 circulars, with no connection to NPCI or Citi systems.

Team: Achilles · Team lead: Sanjay R · 24z241@gmail.com · Source in `03_Source_Code/`

UPI moves 802 million payments a day. In 2025, status checks brought it down three times.

When a bank slows down, payments time out and their outcome becomes **unknown**. Apps ask the switch "did it go through?" and users retry. **Those safeguards become the load.** NPCI said so publicly.

THE SCALE AT STAKE

24.07 bn
UPI payments in Sep 2026, worth **Rs 29.37 lakh crore**

802 M
payments per day on average, a record

≈9,280
payments per second on average (802 M ÷ 86,400 s)

≈557k
payments in one average minute (802 M ÷ 1,440)

RBI turnaround rules (RBI/2019-20/67)

Debit without credit (P2P): the beneficiary bank auto-reverses by **T+1**.
Merchant payment, no confirmation: reversal by **T+5**.
After that, **Rs 100/day** compensation, paid without a complaint.

NPCI technical declines

Declines from connectivity, switch unavailability or bad message data. NPCI counts an incident when **>3 lakh declines last >30 min**.

2025: THREE OUTAGES, THEN THREE RULE CHANGES

26 MAR · 2 APR · 12 APR 2025 — UPI outages

Three nationwide disruptions in 18 days. Payments time out, apps keep asking, and the switch saturates.

APR 2025 · CIRCULAR UPI-OC-215

> NPCI's stated cause: a **"high number of Check Transaction Status APIs at very high TPS by PSP banks"**.

NEW RULE	VALUE
First status check	**≥ 90 s** after the payment (45–60 s after a later revision)
Checks per payment	**at most 3 within 2 hours**
After listed error codes	**stop checking** (e.g. connection timeout, no route to host)
Then	settlement files, or one UDIR check

MAY 2025 · PEAK-HOUR LIMITS

Peak hours are **10:00–13:00** and **17:00–21:30**: members restrict non-customer-initiated APIs. Balance enquiry is capped at **50 per app, per customer, per day**.

16 JUN 2025 · CIRCULAR UPI-OC-214: FASTER TIMEOUTS

API	BEFORE	AFTER
Request / Response Pay	30 s	**15 s**
Check Transaction Status	30 s	**10 s**
Reversal · Validate Address	30 s · 15 s	**10 s · 10 s**

NPCI cautioned that this must not raise technical declines.

Our honest baseline: universe **U2** applies these OC-215 timers in a scaled analogue. We compare QueueShield against **NPCI's own fix**, not just against doing nothing.

WHO FELT AN UNKNOWN PAYMENT

- **Payer.** Money may be debited, but the status is "pending". They retry blind, and may pay twice. If debited, they wait up to T+1 for a reversal.
- **Merchant.** Can't release goods. A no-confirmation payment can take up to T+5 to reverse.
- **PSP / app.** Can only poll the switch. Sees "errors are up", not which bank is sick.
- **Banks.** Already know the outcome, yet answer the same question again and again through the switch.
- **NPCI switch.** Status traffic competes with real payments. Under stress, the questions *are* the outage.
- **Regulator.** Rule compliance and TAT breaches are self-reported and audited yearly, not provable per payment.

SIZE OF THE EXCEPTION PATH (OUR ARITHMETIC)

IF UNKNOWN RATE IS	UNKNOWNS / DAY	AVG / S
0.24% (our S6 runs)	≈1.9 M	≈22
0.5% (assumed)	≈4.0 M	≈46

802 M/day × rate. Each one, without control, can trigger ~18 status checks (U0, S6, sim).

MISSING TODAY — What timers can't give you

- **An early warning.** Nothing says a slowdown is about to become a **self-sustaining** storm while there is still time to act.
- **Localisation.** Is it a PSP, a bank, a route or the switch? Failing over across a shared bank **spreads** the fault.
- **A shared truth.** No common, signed, **push-based** record of payment state that every party can subscribe to.
- **A network-wide budget.** Timers are honoured bank by bank and checked by **annual audit**. No live load-aware budget exists.

Sources: NPCI data via StartupTalky (Sep 2026) · MediaNama on NPCI UPI-OC-215 (30 Apr 2025), peak-hour guidelines (May 2025) and UPI-OC-214 (17 Jun 2025) · RBI/2019-20/67 TAT circular · NPCI BD/TD & uptime statistics. Per-second and per-minute figures are our arithmetic.

The storm loop: every safeguard adds load to the system that is already failing

Checks and retries are meant to help. Under stress they feed a loop that can **outlive its trigger**: a **metastable failure**. QueueShield is built to break this loop at four points.

THE FEEDBACK LOOP



Break ① detect & localise early (M3)

Break ② resolve unknowns by push (M1)

Break ③ budget checks, slow retries (M4)

Break ④ never shift into a shared bank

ANATOMY OF ONE UNKNOWN PAYMENT UNDER THREE POLICIES

S6 · SIM · MEDIAN OF 5

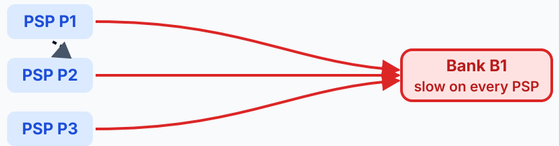
PHASE	U0 · NO CONTROL	U2 · NPCI RULE (OC-215)	U4 · QUEUESHIELD
Times out	Client gives up at **15 s** (OC-214) → outcome **UNKNOWN** · 3 s in our 5:1 time-compressed analogue		
First check	immediately, then **every 2 s**	≥ 90 s later (10 s analogue)	waits for the ledger push first (`ledger_wait` 10 s)
Check limit	**none**	3 within 2 h, then stop	3, released by a **token bucket** sized from spare switch capacity
Resolved by	the switch, eventually	the switch, or **escalated** to manual handling	the **bank's signed attestation**, pushed from Drunix
Checks per unknown	**18.4**	**3.0**	**7.2** (30% of clients stay naive)
Escalated to manual	0 (never stops asking)	**235**	**166 (−29%)**

WHAT THE SWITCH ACTUALLY SERVES

```
load = fresh payments
     + retries × RAF
     + status checks
     + zombie work
```

RAF: retry amplification factor (retries ÷ fresh attempts). **Zombie work**: requests still served after the client gave up. In our model the switch slows past **85% utilisation** (the knee).

THE SHARED-BANK FAILOVER TRAP



Moving P1 → P2 still lands on B1. In our simulator, every S4 run where textbook failover (U1) moved P1's traffic **collapsed the whole network**: goodput afterwards fell to **0/s in U1**, against 18/s with no control SIM.

THIS IS A KNOWN FAILURE CLASS, SO WE BUILT ON THE RESEARCH

Huang et al., OSDI 2022. 22 metastable failures from 11 organisations. Retry policy is the most common sustaining effect (>50%). At least 4 of 15 major AWS outages in a decade were metastable.

Isaacs et al., HotOS 2025 (AWS). Combine queueing models with discrete-event simulators to find where systems are vulnerable. That is our method: M2 plus a lock-in forecaster.

Google SRE, "Handling Overload". Retry budgets: ≤ 3 attempts per request and ≤ 10% retry ratio per client keep growth near ~1.1× instead of ~3×. Our check budgets apply the same idea to status checks.

Why dashboards see it late: the platform success rate is an average over 800 M payments. **Leading signals move first**, on one segment:

`latency ratio` `near-timeout share` `timeouts` `retry ratio (RAF)` `queue growth` `pending unknowns`

Sources: MediaNama on UPI-OC-214/215 · Huang et al., "Metastable Failures in the Wild", OSDI 2022 · Isaacs et al., "Analyzing Metastable Failures", HotOS 2025 · Google SRE book. Policy numbers come from our S6 batch runs (seeds 1–5, synthetic data). Real timers are compressed in the simulator (labelled analogue).

Today's fixes act per payment or per bank. Nobody manages the storm as a network.

We compared the published rules, common industry patterns and the public Drunix hackathon projects we could find. QueueShield **combines** the per-payment truth they get right with network-level detection, forecasting and budgets.

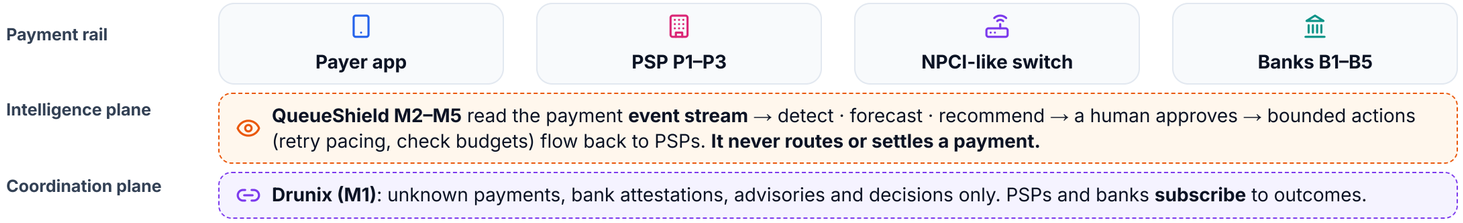
CAPABILITY MATRIX: OUR READING OF PUBLIC DESCRIPTIONS

APPROACH	NAMES THE SICK SEGMENT	FORECASTS LOCK-IN	BUDGETS CHECKS NETWORK-WIDE	PUSHES OUTCOMES OF UNKNOWNS	SHARED SIGNED AUDIT TRAIL	HUMAN-APPROVED ACTIONS	AVOIDS SHARED-BANK FAILOVER	MEASURED VS NPCI RULE
NPCI OC-215 timers 90 s first check, ≤3 in 2 h	✗	✗	◐ fixed, per PSP	✗	✗ annual audit	—	—	—
NPCI OC-214 timeouts 30 s → 10–15 s	✗	✗	✗	✗	✗	—	—	—
Success-rate failover textbook reactive rule	◐ PSP only	✗	✗	✗	✗	✗ automatic	✗ spreads it	—
Gateway bandit routing Thompson sampling (Cashfree blog)	◐ per gateway	✗	✗	✗	✗	✗	◐ gradual	—
Pulse (Drunix entry) per-bank signed transitions, SLA → STUCK	✗	✗	✗	✓	✓	✗	—	✗ self-reported
Tally (Drunix entry) proposal: shared history, idempotency	✗	✗	✗	◐ proposed	◐ proposed	✗	—	✗
Citi Token Services own permissioned chain	—	—	—	✓ instant status notifications	✓ within Citi	—	—	—
QueueShield × Drunix this entry	✓ PSP · bank · route · switch	✓ p_stuck, AUC 0.93 SIM	✓ load-aware token bucket	✓ Drunix events	✓ attests, advisories, decisions	✓ one key, recorded	✓ locked in UI + code	✓ paired seeds vs U2

✓ does it ◐ partly ✗ does not — not its job

Competitor entries read on GitHub on 20 Oct 2026. Pulse reports ~90–95% fewer status requests (self-reported).

WHERE QUEUESHIELD SITS: BESIDE THE RAIL, NEVER IN THE PAYMENT PATH



OUR POSITION (CLAIMS-SAFE)

Among the public Drunix projects we reviewed, QueueShield is the one that treats unknown payments as a network-level load problem. It detects the tipping segment, forecasts lock-in, budgets verification, and records the operator's decisions on the same ledger.

IDEAS WE DELIBERATELY BORROWED

- **Citi Token Services:** push status notifications instead of polling. We apply the same design principle to UPI exceptions.
- **Citi Payment Outlier Detection:** the model flags and the client approves or rejects before release. That is our human-in-the-loop pattern.
- **Google SRE retry budgets**, and **adaptive LIFO + CoDel** (Facebook, HBase): under overload, cap retries and drop stale work first. These became `THROTTLE_RETRIES`, `SHED_EXPIRED` and `PRIORITIZE_FRESH`.
- **Pulse / Tally:** one signed truth per payment. Included as our `paystate` contract, then extended with advisories and decisions.
- **Project Axis** maker-checker: our planned build-phase rule that a traffic shift needs a second approver.

ORIGINALITY DISCLOSURE

QueueShield's detection core (windows, baselines, rule signals, lifecycle) began in our earlier **IBM Z Datathon** entry. Every reused part is listed in `docs/REUSE_MAP.md`. The **Drunix ledger, UPI network model, localiser, switch features, forecaster training and this build are new**.

"ISN'T THIS JUST PULSE?"

No. They resolve **one payment** honestly, and we include that pattern. We add the **network level**: find the tipping segment, forecast lock-in, budget the checks, and show the system effect on **paired seeds against the NPCI rule**.

Sources: github.com/cs25b1074-stack/citi (Pulse) · github.com/sharannya310106/tally-drunix · github.com/ferroflux/project-axis · services.citi.com Citi Token Services · Citi Payment Outlier Detection press release (2019) · Cashfree engineering blog · Google SRE book · HBase AdaptiveLifoCoDelCallQueue. Matrix cells are our judgement from public material.

One payment during a bank slowdown, told twice: today, and with QueueShield

Same fault: bank **B1** slows on every PSP. Times in the lower lane are from our recorded S3 simulation run, which is the replay shown in the demo video SIM.

Today polling and blind retries	t = 0 PAYER Taps **Pay Rs 1,200** at a store. The app sends a pay request through the switch.	t = 15 s PSP APP B1 is slow, so the request **times out**. Debited? Credited? **UNKNOWN**.	t = 15 s + PSP → SWITCH Apps **poll the switch** again and again. Without control that is 18.4 checks per unknown SIM.	≈ 1 min PAYER Sees "pending" and **retries**. The second attempt goes to the same slow bank, a possible **double debit**.	minutes NPCI SWITCH Status traffic competes with payments. **Healthy banks slow too**, and the loop sustains itself.	minutes OPS Dashboard: "**success rate down**". Which bank? Failover to another PSP still hits B1.	T+1 day BANK → PAYER If debited: auto-**reversal by T+1**, then Rs 100/day. The merchant has already sent the customer away.
With QueueShield push, localise, approve	sim 150 s FAULT Bank **B1**'s legs slow ×4–8 on **every** PSP (scenario S3). Payments start timing out.	same tick M1 · DRUNIX The PSP opens each unknown on the ledger: `paystate:OpenUnknown`. The **TAT clock starts**.	sim 165 s · +15 s M3 · DETECT The localiser says **BANK_SIDE · B1**: B1's column lights up for all three PSPs.	sim 165 s M4 · CONTROL Recommends **slow retries + defer checks**. Failover is **blocked**: "shared bank B1 is slow on every PSP".	sim 176 s M5 · OPERATOR **Approves with one key.** The decision and its `DO_NOT_SHIFT` advisory are written to the ledger with a tx id.	seconds BANK → ALL PSPS B1 **attests DEBIT/CREDIT once**. A `PaymentUpdated` event is **pushed** to every PSP. No polling needed.	done PAYER Sees a **final status**, so there's no blind retry. If the bank misses its TAT, a `TATBreach` is recorded once.

On Drunix
what the ledger holds
	nothing yet: healthy payments never touch the ledger	`PAY~run:p-0000912`
		state **UNKNOWN**, openedAt, version 1
			detection is off-ledger (M3): no write	a recommendation stays off-ledger until it is decided	`DEC~dec-00001`
					`ADV~adv-00001`
					`DO_NOT_SHIFT`
					evidenceHash = sha256(rec)
						UNKNOWN → DEBITED → SUCCESS; tatDeadline = debitAt + TAT
						`event PaymentUpdated`
							terminal: SUCCESS · FAILED_NO_DEBIT · REVERSED
							`event TATBreach`

Payer

Needs: to know if the money moved.
Gets: a final status pushed from the bank's attestation, so no second payment.
We measure: duplicate commits; median and p95 time to resolve.

PSP / app ops

Needs: answers without polling.
Gets: outcomes by subscription, a live check budget, and named segments in alerts.
We measure: switch checks per unknown; peak checks/s.

Banks

Needs: fewer repeated status queries.
Gets: attest a leg **once**, signed by their own org, which is less work than answering the same query repeatedly.
We measure: attestations per unknown; harmful failovers.

NPCI network ops

Needs: to know which segment is tipping, and how soon.
Gets: a verdict, a p_stuck forecast, and bounded actions to approve.
We measure: detection delay; runs locked into a storm.

Regulator

Needs: provable rule compliance and TAT.
Gets: one signed history of attestations, advisories, decisions and breaches. A read-only view is planned.
We measure: TAT breaches; decisions with tx ids.

Sponsor bank (a Citi-like role)

Needs: to pay out bulk batches without hitting a sick bank.
Gets: early warning on beneficiary banks and `PACE_PAYOUTS` (scenario S8).
We measure: payout goodput into a slow bank.

Notes: the "Today" lane follows the loop NPCI described in UPI-OC-215; T+1 is from RBI/2019-20/67. The QueueShield lane uses times from recording `s3_bank` (fault 150 s → BANK_SIDE 165 s → approval 176 s, simulation clock). The sponsor-bank row is a role in our simulator, not a Citi integration.

Detect, forecast, recommend, approve, resolve: five steps that break the loop

QueueShield is an **operator decision-support and coordination layer** with two planes. An **off-ledger intelligence plane** watches every payment event. An **on-ledger coordination plane** on Drunix holds only what several organisations must agree on.

1 · Detect & localise

M3 · qdet

INPUT
payment events (SSE), 1 s windows, 25 segments

HOW
23 features with EWMA baselines. The D3 rules use 6 signals plus a **surge gate** and 3-of-4 persistence. The **involvement-lift localiser** gives 6 verdicts.

OUTPUT
`incident`: segment, verdict, severity, confidence, reasons

S3 bank slowdown: named **BANK_SIDE · B1** 15 s after the fault began SIM

2 · Forecast lock-in

M3 · forecaster

INPUT
per-PSP queue delay ÷ timeout, retry backlog ÷ capacity

HOW
a 20×20 monotone grid learned from simulation gives **p_stuck**: will it lock in *if nothing changes*? EMA smoothing, CALM / WATCH / CRITICAL, rough countdown.

OUTPUT
severity CRITICAL; orders the recommendations

Held-out validation: AUC **0.93**; at the 0.6 threshold, 0 false criticals, 9/14 stuck runs warned before collapse SIM

3 · Recommend & budget

M4 · qctl

INPUT
incidents, verdicts, switch utilisation

HOW
rules **R0–R7** pick bounded actions. Eligibility checks **block unsafe failover**. The **Verification Governor** sizes a per-PSP check budget from spare switch capacity.

OUTPUT
`recommendation` with side effects and rollback conditions; `CHECK_BUDGET`

9 safety invariants in code, e.g. never re-attempt an unknown payment

4 · Human approves

M5 · console

INPUT
a pending package: actions, side effects, confidence, p_stuck

HOW
the operator ticks eligible actions and approves with **one key**. Blocked actions are locked and **can never be sent**. Rejection is recorded too.

OUTPUT
`decision` → actions for the network → **ledger record**

S3 replay: approved 11 s after the bank was named; decision tx shown in the log SIM

5 · Resolve by push

M1 · Drunix

INPUT
unknown payments, attestations, advisories, decisions

HOW
the bank that knows **attests once**, signed by its org. A chaincode event is **pushed** to every PSP. RBI TAT clocks run on-ledger.

OUTPUT
`PaymentUpdated`, `AdvisoryIssued`, `DecisionRecorded`, `TATBreach`

S6 replay: switch checks per unknown **18.4** → **7.2** vs no control SIM

TEN DESIGN PRINCIPLES EVERY MODULE MUST RESPECT

1. **Localise before you act.** Every alert names a segment and a level.
2. **Leading indicators beat success rate.** Latency, near-timeouts, retry ratio and queue growth move first.
3. **Separate surge from degradation.** A surge has stable latency and draining queues.
4. **A timeout is unknown, not failure.** Resolve by evidence before any re-attempt.
5. **Safeguards are load.** Status checks, retries and shifted traffic are all budgeted.
6. **Push, don't poll.** The bank attests once; everyone else subscribes.
7. **Ledger for the exceptions**, not the firehose: about 0.25% of payments SIM.
8. **Right-size the response.** Slow retries first, defer checks next, shift traffic last and gradually.
9. **Human in the loop.** The system recommends; a person approves; the approval is on the ledger.
10. **Honest evaluation.** Identical seeds, a strong NPCI-rule baseline, negative results reported.

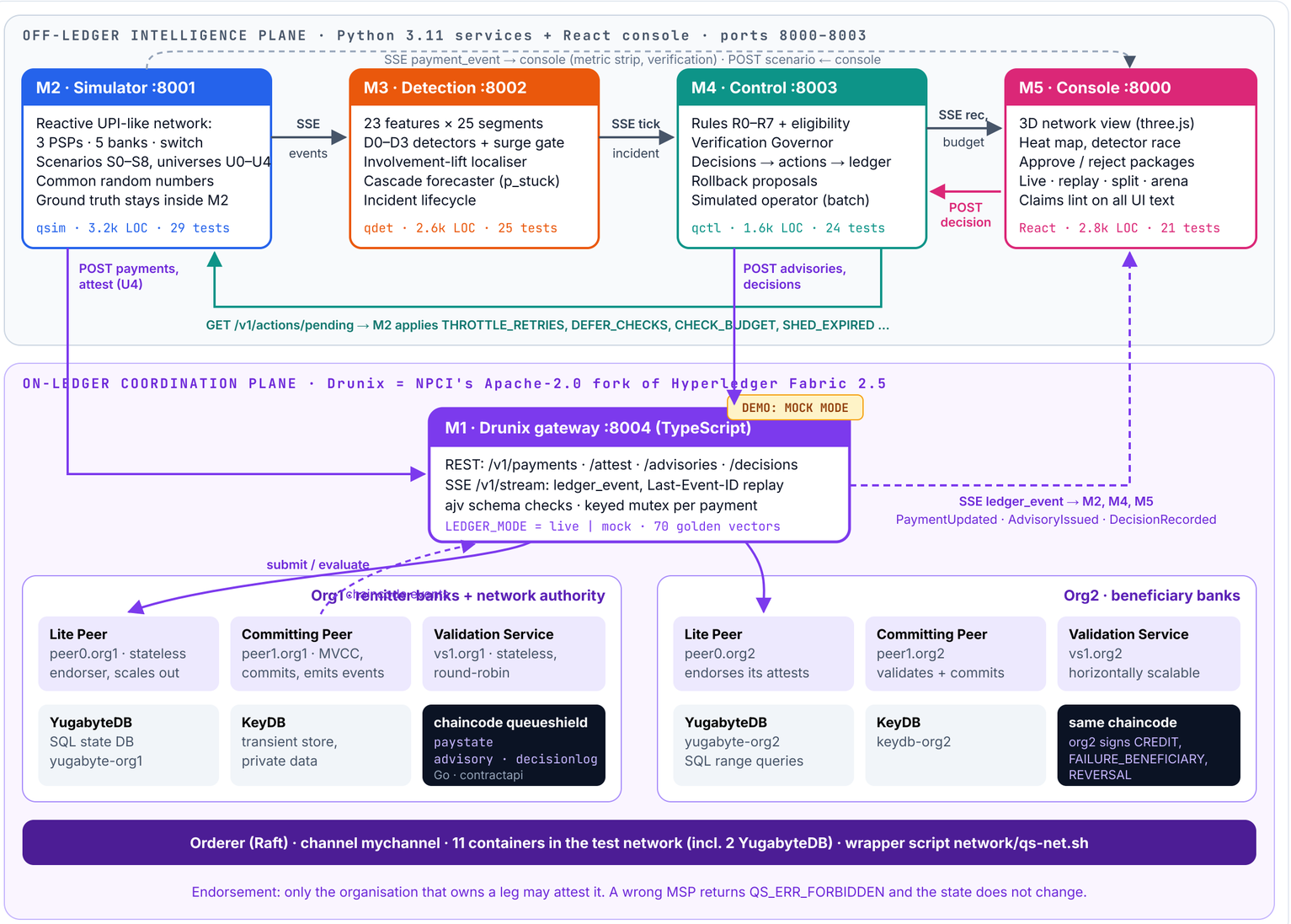
WHAT QUEUESHIELD IS, AND IS NOT

✓ IS	✗ IS NOT
A detector and decision aid for operators	An autonomous router or a payment processor
A forecaster of whether a degradation **locks in if nothing changes**	A predictor of faults before they begin
A shared, signed record for **exceptions**	A ledger for every UPI payment
Evaluated in a reactive simulator, on paired seeds	Validated on real bank traffic
Built for a Drunix network (chaincode + gateway)	NPCI-approved or Citi-integrated
Explainable: every recommendation stores model versions, a features hash, reasons and confidence	A black box, or an LLM in the decision path
Supporting evidence for a human ("reasons are not proof of cause")	Automatic action on money or routing

Evidence: detection delay from recording s3_bank · forecaster figures from `data/models/forecaster_report.json` (validation set B: 30 runs, 14 stuck) · S6 figures are medians of seeds 1–5. All simulation, synthetic data.

Five independent modules, two planes, one contract folder. No module imports another.

Modules talk only over **HTTP + Server-Sent Events**, validated against **17 versioned JSON Schemas**. Each runs alone against fixtures, so five people built in parallel. Only exceptions cross into the ledger.



CONTRACTS/SCHEMAS: THE ONLY SHARED CODE

payment_event	segment_state
incident	recommendation
decision	action
advisory	ledger_event
ledger_open_request	ledger_attest_request
ledger_payment	ledger_decision
lockstep	run_result
scenario_command	network_info
common	+ fixtures, PROPOSALS/

Changes go through `contracts/PROPOSALS/`. Invalid messages are dropped and counted, and never crash the receiver.

THREE RUN MODES

MODE	CLOCK	LEDGER	USED FOR
live	M2 wall clock ×1/2/4	Drunix or mock	demo, integration
mock	each module alone	mock	dev, contract tests
batch	lockstep, 5 s ticks	latency model	arena, training

FAILURE ISOLATION

Ledger down	M2 falls back to its model; console shows "ledger offline", **never fakes events**
Detection down	no incidents, so M4 recommends nothing
Control down	M2 keeps the last applied actions
Model slow	> 50 ms twice → rule signals only, DEGRADED_MONITORING
SSE gap	ring buffer ≥ 50k msgs, Last-Event-ID resume, 5 s heartbeat

SECURITY BY DESIGN

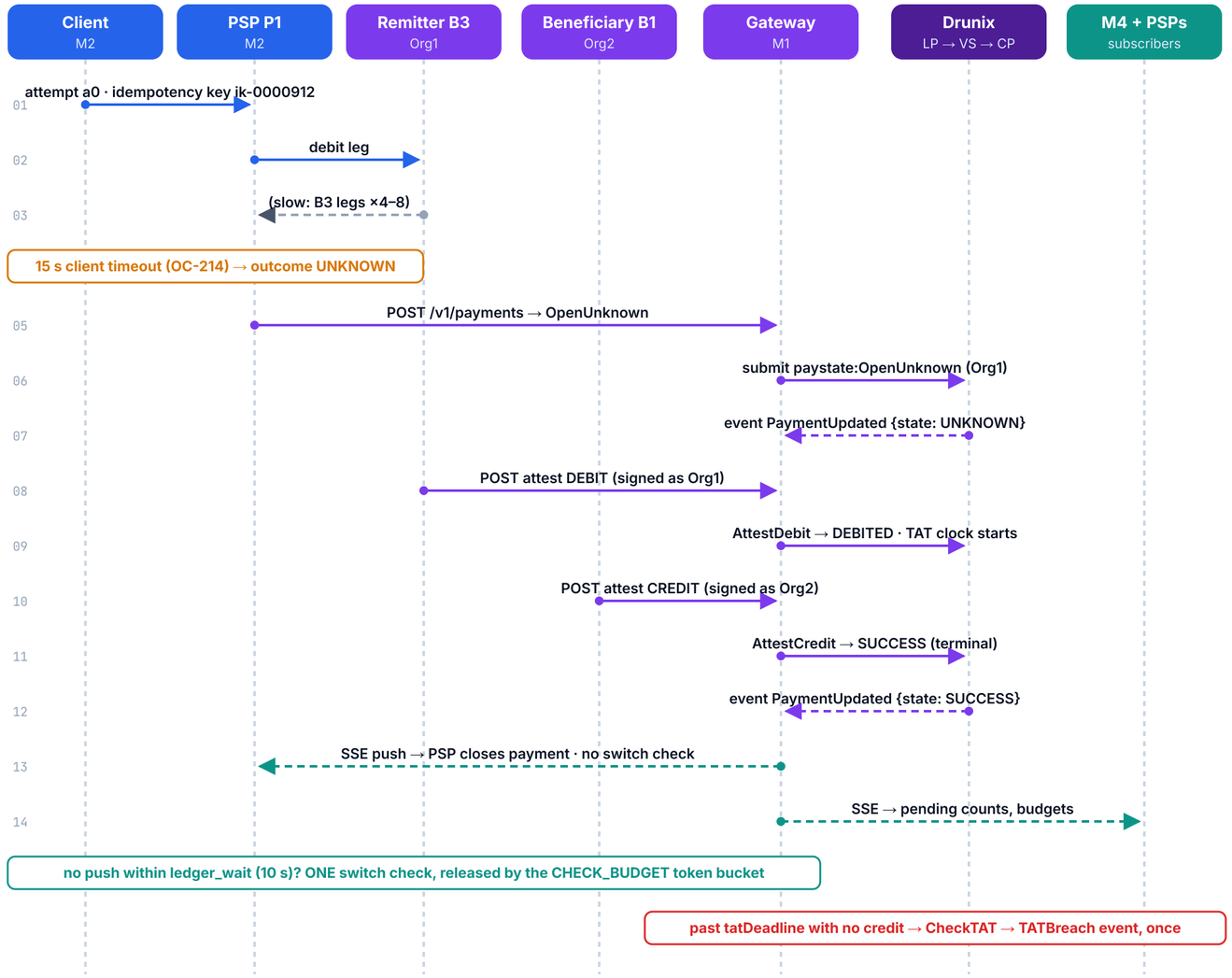
- Every write needs header `X-QS-Key`; reads are open on localhost only
- Ledger payloads carry **no amounts, accounts, VPAs or names** (schema-enforced, SI-10)
- Idempotency keys are stored as `sha256` hashes

From the repo: docs/01_SYSTEM.md (topology, ports, run modes, failure isolation) · contracts/schemas (17 files) · line counts are Python/TS/Go source per module from `git ls-files` · test counts from the last runs (106 total, incl. M1 gateway 7).

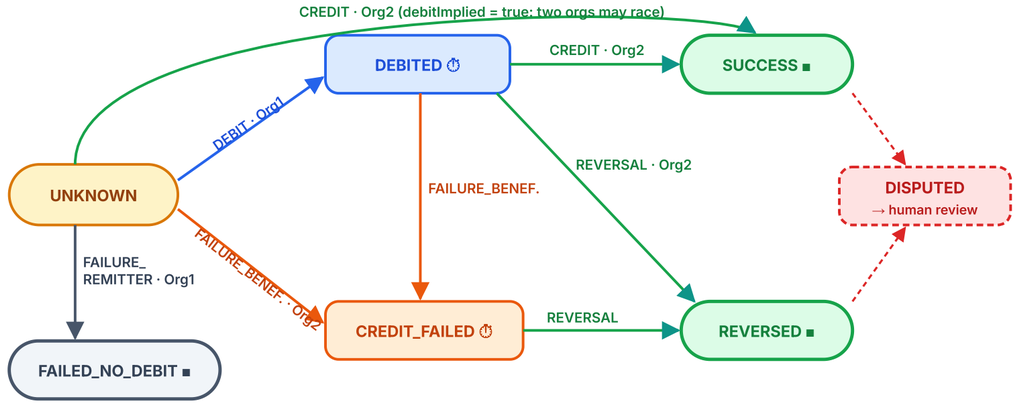
Life of an unknown payment on Drunix: opened once, attested by its owner, pushed to all

One Go chaincode, `queueshield`, with three contracts. A **pure state machine** (no Fabric imports) is mirrored in TypeScript and tested against the same **70 golden vectors**. Only the organisation that owns a leg may attest it.

SEQUENCE: U4 UNIVERSE, LIVE MODE (THE DEMO USES THE MOCK GATEWAY WITH THE SAME CALLS)



PAYSTATE STATE MACHINE (TABLE 4.3 → TESTDATA/TRANSITIONS.JSON)



■ terminal

⏱ RBI TAT clock running: `tatDeadline = debitAt + tatSeconds`. Past it, `CheckTAT` emits **TATBreach exactly once**.

Duplicates are no-ops. DISPUTED appends attestations and is flagged for humans.

THREE CONTRACTS, ONE CHAINCODE

CONTRACT · KEY	FUNCTIONS	WHO MAY WRITE	EVENTS
paystate `PAY~run:payment`	OpenUnknown · AttestDebit · AttestCredit · AttestFailure · AttestReversal · GetStatus · CheckTAT	PSP opens (idempotent on idemKeyHash); **the owning org** attests each leg	PaymentUpdated · TATBreach
advisory `ADV~adv-NNNNN`	IssueAdvisory · RevokeAdvisory · GetAdvisory · ListActive	authority MSP; time-boxed 5–3,600 s; evidenceHash	AdvisoryIssued · AdvisoryRevoked
decisionlog `DEC~dec-NNNNN`	RecordDecision · GetDecision	authority MSP; **append-only**, id unique	DecisionRecorded

4 ADVISORY KINDS

SLOW_RETRIES ← THROTTLE_RETRIES
DEFER_CHECKS ← DEFER_CHECKS
DO_NOT_SHIFT ← a blocked SHIFT_TRAFFIC
PACE_PAYOUTS ← PACE_PAYOUTS
Every advisory carries the decision id. No advisory exists without a decision (SI-9).

VERIFIED SO FAR

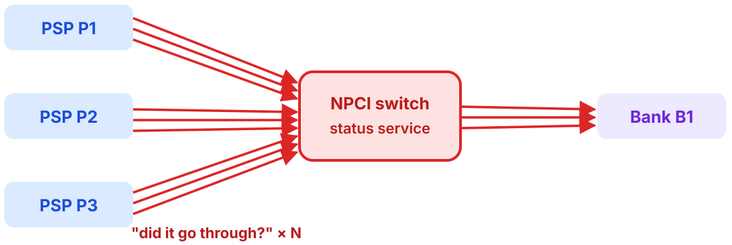
- ✅ TS state machine passes all **70 golden vectors**
- ✅ **100 concurrent** DEBIT+CREDIT pairs: no lost attestation
- ✅ SSE ring buffer, Last-Event-ID replay, heartbeat
- ✅ M2 bridge: unknowns opened and attested through the real mock gateway
- 🟡 Go chaincode written, **not yet run** (no Go on our dev machine)

From the repo: m1_ledger/SPEC.md §4 (state table, chaincode API) · m1_ledger/STATUS.md (tests L3–L8) · docs/01_SYSTEM.md §5 (sequence) · Drunix: github.com/npci/drunix, docs/drunix-arch.md. Live Drunix bring-up is our first build-phase task.

Push, don't poll: the bank that knows attests once, and everyone is told

The truth about an unknown payment is **split across organisations**: the remitter knows the debit, the beneficiary knows the credit. A permissioned ledger lets each sign **only its own fact**, and lets every party subscribe to the result.

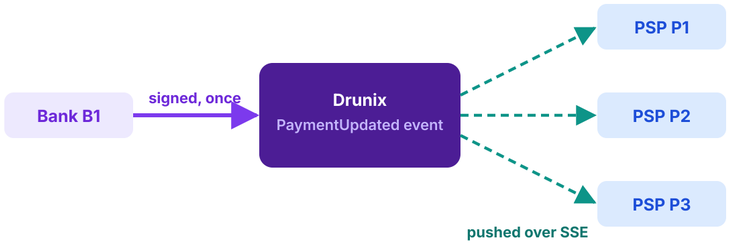
TODAY: POLL THE SWITCH



18.4
switch checks per unknown, U0
S6 · SIM

2,570/s
peak checks in a storm run, seed 3 SIM

QUEUESHIELD: ATTEST ONCE, PUSH TO ALL



7.2
checks per unknown, U4 (−61%)
S6 · SIM

38/s
peak checks, same seed 3, same fault
SIM

LEDGER FOR EXCEPTIONS, NOT THE FIREHOSE

≈97,000 payments in one S6 run: stay **off-ledger** (event stream only) SIM

↓

≈235 unknowns (0.24%) are opened on Drunix

↓

≤ 2 attestations each, from the owning banks

↓

a handful of advisories and decisions

BACK-OF-ENVELOPE AT UPI SCALE (OUR ARITHMETIC)

UNKNOWN RATE	UNKNOWNS/DAY	UNKNOWNS/S	LEDGER TX/S*
0.24% as in our S6 runs	≈1.9 M	≈22	≈67
0.5% assumed	≈4.0 M	≈46	≈139

802 M payments/day. *1 open + 2 attestations per unknown, daily average; peaks are higher.

Not measured yet: Drunix throughput on our chaincode (hypothesis P8, a build-phase benchmark at 10/25/50 tx/s). Drunix is designed to scale: **Lite Peers and Validation Services are stateless and scale horizontally**, and state lives in YugabyteDB.

PRIVACY BY CONSTRUCTION

Ledger records hold ids, banks, PSPs, states and timestamps only. **No amounts, accounts, VPAs or names**: the schema rejects them (SI-10). Idempotency keys are stored as `sha256`.

WHY A BLOCKCHAIN, NOT A SHARED DATABASE?

- **Each fact is signed by the org that knows it.** The endorsement policy enforces who may say what: Org1 debits, Org2 credits and reversals.
- **Tamper-evident, append-only history.** No single operator, including us, can rewrite an attestation or a decision.
- **Compliance becomes provable.** The chaincode checks TAT against its own transaction time, so a stale query can never cause a false breach.
- **No central owner to trust.** PSPs, banks and the authority share one record, and a regulator can read it.

WHY DRUNIX SPECIFICALLY

- **NPCI's own fork** of Hyperledger Fabric 2.5, Apache-2.0, backward compatible with HLF 2.5.x
- **SQL state DB** (YugabyteDB): we query TAT candidates and per-state counts in SQL
- **Lite Peer / Committing Peer split**: endorsement scales out, commits stay consistent
- **KeyDB transient store**: room for bank-private data later

Same principle as Citi Token Services: instant, API-initiated payment status notifications from a permissioned chain, instead of clients polling. No connection to Citi systems.

"Do banks change anything?" They attest the legs of unknown payments they already know about. That is less work than answering repeated status checks.

WHO SIGNS

what, on the ledger

PSP
opens the unknown (`OpenUnknown`), idempotent on the key hash

Remitter bank · Org1
`DEBIT` , `FAILURE_REMITTER`

Beneficiary · Org2
`CREDIT` , `FAILURE_BENEFICIARY` , `REVERSAL`

Network authority
advisories and operator decisions, linked by decision id

Regulator
read-only view of TAT breaches and decisions (planned, build phase)

Everyone
subscribes to chaincode events through the gateway's SSE stream

Sources: S6 batch runs (seeds 1–5; seed 3 for the split) · github.com/npci/drunix and docs/drunix-arch.md · services.citi.com Citi Token Services · m1_ledger/SPEC.md. UPI volume: NPCI via StartupTalky (Sep 2026). Scale rows are our arithmetic, not a benchmark.

Detection and localisation: 25 segments, 6 leading signals, 6 verdicts, every 5-second tick

Every tick (5 s), M3 scores every PSP, bank, route and the switch against its **own learned baseline**, separates a **surge** from a **degradation**, then asks which dimension explains the excess failures.

① Ingest
schema check, LRU dedupe, **2 s watermark**; late events still land

② Windows
1 s buckets, short + long windows, latency histograms

③ Features
23 per segment; NaN is never treated as 0

④ Baselines
EWMA mean / MAD, warm-up, **frozen while an incident is open**

⑤ D3 rules
6 signals → logistic score; **surge gate** annotates instead of alerting

⑥ Persistence
WARNING when score ≥ τ in **3 of 4 ticks** with ≥ 2 signals

⑦ Localiser
involvement lift → verdict; must repeat for **2 ticks**

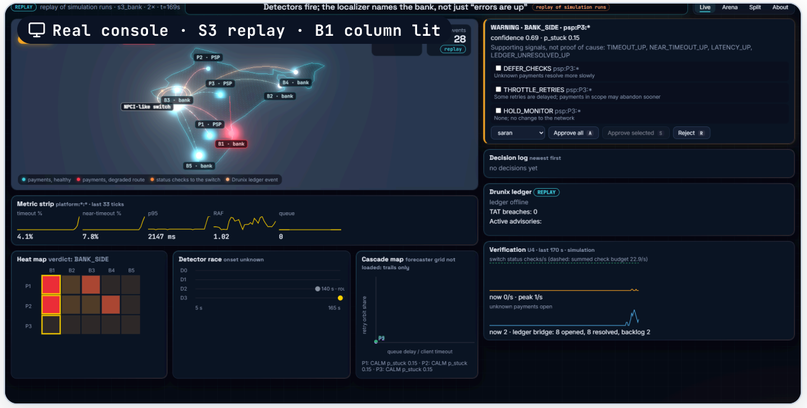
⑧ Incident
OPEN → MITIGATING → RECOVERING → CLOSED, merged by primary

25 SEGMENTS WATCHED EVERY TICK

`platform ×1` `PSP ×3` `bank ×5` `route ×15` `switch ×1`

	B1	B2	B3	B4	B5
P1	0.41	.04	.01	.00	.01
P2	0.33	.01	.03	.00	.01
P3	0.12	.01	.00	.01	.01

Illustration of the route-cell share of excess failures. A bank fault **lights a column**, a PSP fault lights a row, and a route fault lights one cell. A bank counts in **both roles**, remitter and beneficiary.



Result `SIM`**:** bank B1 slowed at sim 150 s. **BANK_SIDE · B1** at 165 s (+15 s). The same simulator's festive surge (S1 recording) raised **no incident**.

D3: SIX LEADING SIGNALS (BOOLEANS PER TICK)

SIGNAL	FIRES WHEN
`lat` latency	z(latency ratio) ≥ 2.5
`to` timeouts	timeout ratio ≥ mean + 3·spread and ≥ 2% absolute
`near_to`	near-timeout ratio ≥ mean + 3·spread and ≥ 5%
`q` queue	queue growing and delay ≥ 1.5 × baseline
`raf` retries	retry amplification ≥ 1.25 × baseline
`rej` rejects	reject ratio ≥ mean + 3·spread and ≥ 1%

SURGE GATE: A FESTIVAL IS NOT A FAULT

SURGE (annotation, no alert) if arrivals z ≥ 2.5, **≤ 1** signal true and the queue is **not growing**, for 2 ticks. Principle 3: surge = more traffic, stable latency, draining queue.

SCORE, SEVERITY, CONFIDENCE

```
score = logistic(true_signals / 6)
WARNING: score ≥ τ in 3/4 ticks ∧ ≥2 signals
fast path: score ≥ 0.999 ∧ ≥3 signals, 2 ticks
CRITICAL: WARNING ∧ p_stuck ≥ 0.5
conf = .4·score + .3·sig/6 + .2·persist + .1·n
```

Scoring > 50 ms twice → `model_degraded`: rule signals only, DEGRADED_MONITORING (tested).

LOCALISER: WHICH DIMENSION EXPLAINS THE EXCESS?

```
N_ex = max(0, N_bad - N_expected) ≥ 20
EP[p] = PSP p's share of excess bad
IB[b] = share of excess bad involving bank b
lift[b] = IB[b] - baseline involvement of b
```

VERDICT	CONDITION (START VALUES)	SCENARIO
BANK_SIDE(b)	IB ≥ .75, lift ≥ .4, other IB ≤ .5, max EP < .6	S3, S6, S8
PSP_SIDE(p)	EP ≥ .67, max lift < .3	S2, S4
ROUTE_SPECIFIC	EP ≥ .67, IB ≥ .75, cell ≥ .4	S7
SWITCH_STATUS_OVERLOAD	switch util ≥ .9 for 2 ticks, no PSP/bank dominant	S6
MULTI_FAULT	two banks IB ≥ .6, or two PSPs EP ≥ .35	S5
PLATFORM_WIDE	otherwise, with N_ex ≥ 20	S1 overload

Tested (D-6): each of the 6 verdicts recovered in **≥ 95%** of 1,000 Poisson trials.

REASONS SHOWN TO THE OPERATOR

`TIMEOUT_UP` `NEAR_TIMEOUT_UP` `LATENCY_UP` `RAF_UP` `QUEUE_GROWING` `REJECTS_UP` `PENDING_BACKLOG` `SWITCH_STATUS_SATURATED` `LEDGER_UNRESOLVED_UP`

Each with observed / baseline / z. The UI says it plainly: **supporting evidence, not proof of cause**.

COMPARISON DETECTORS, RUN EVERY TICK

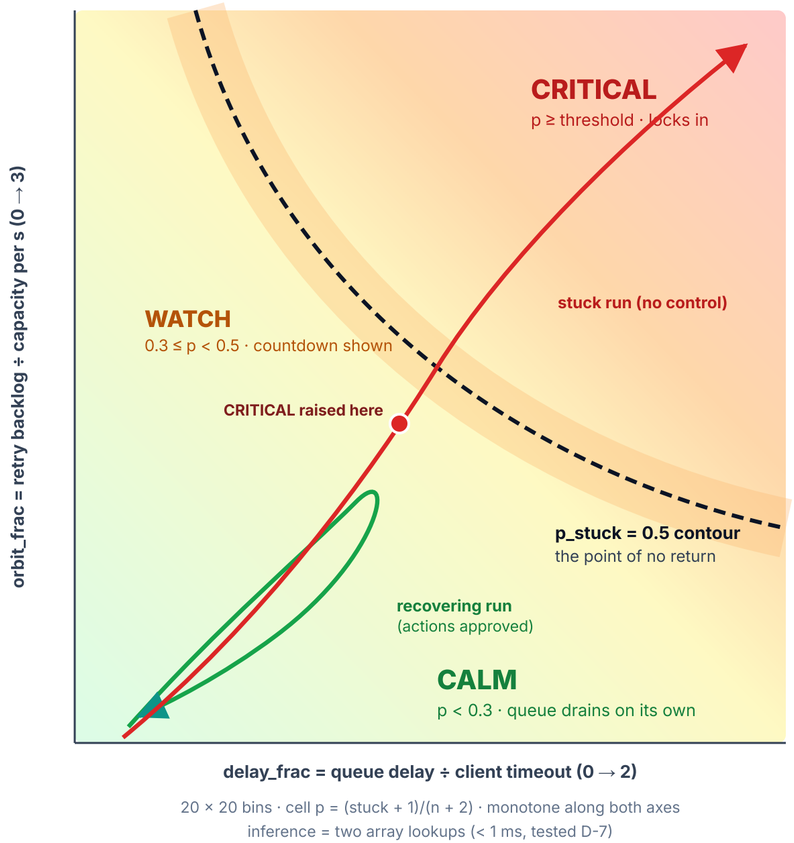
D0 platform dashboard · **D1** static thresholds · **D2** CUSUM · **D3** QueueShield rules. Fairness rule: each is tuned to **≤ 1 false alert per simulated hour** before any delay comparison (build phase).

From the repo: m3_detection/SPEC.md §4–§7 (pipeline, signals, verdict table) · m3_detection/STATUS.md (tests D-1…D-10, robustness) · recordings s3_bank and s1_festive. Heat-map values in the grid are illustrative. D0–D2 thresholds are not yet tuned.

Forecasting lock-in: a learned point-of-no-return map, validated on held-out runs

For every PSP, every tick: the probability **p_stuck** that its queue and retry backlog lock into a self-sustaining collapse **if nothing changes**. It forecasts the cascade, not the onset. It never triggers actions: it raises severity and orders recommendations.

THE PHASE PLANE (SCHEMATIC)



20 × 20 bins · cell p = (stuck + 1)/(n + 2) · monotone along both axes
inference = two array lookups (< 1 ms, tested D-7)

Schematic for explanation: shape and trajectories are illustrative. The trained grid is exported to `data/models/forecaster_grid.json` with its 0.5 contour.

TRAINING AND HELD-OUT VALIDATION SIM

SET	SEEDS	RUNS	STUCK	USE
A	1–999	60	29	fit the grid (S4 + S6, U0 only)
B	1000–1999	30	14	choose the threshold, report
C	2000–2999	—	—	test, run once after freeze (build phase)

0.93
sample AUC on B (monotone grid)

0.077
Brier score on B

14/14
stuck runs flagged at every threshold 0.5–0.9

Labels come from ground truth, which M3 reads **only in offline training scripts**, never at inference. Stuck = goodput < 50% of baseline 60–120 s after the trigger ends.

THE THRESHOLD TRADE-OFF ON SET B SIM

CRITICAL AT	WARNED BEFORE COLLAPSE	MEDIAN LEAD	FALSE-CRITICAL RUNS
p ≥ 0.5	**14 / 14**	**18.5 s**	**9 / 16**
p ≥ 0.6 (chosen)	9 / 14 (64%)	5.5 s	**0 / 16**
p ≥ 0.7 – 0.9	9 / 14	5.5 s	0 / 16



RUNTIME, EVERY TICK, PER PSP

1. Look up p from the grid, then smooth with an **EMA (weight 0.5)**
2. State: `CALM < 0.3` `WATCH` `CRITICAL ≥ threshold`
3. **Countdown**: linear extrapolation of delay_frac over 4 ticks to 1.0, labelled *"rough estimate"*
4. In S6 the switch couples every PSP, so p_stuck is evaluated for **all PSPs**
5. Too slow (> 50 ms twice)? Rule signals only, and the console shows **DEGRADED_MONITORING**

WHAT WE ARE HONEST ABOUT

- **Sample AUC is inflated** by ticks after a collapse. The honest number is **lead time**: at our chosen 0.6, only 9 of 14 stuck runs were warned before collapse, with a median lead of 5.5 s.
- The trained grid is **too coarse to draw**: 99.8% of its cells are ≥ 0.5, so the console's cascade map shows trails only and does not load it. This is logged in DEVIATIONS.
- Trained on **U0 simulator runs**, not real traffic.

WHERE P_STUCK GOES

- WARNING + p_stuck ≥ threshold → **CRITICAL**, which unlocks rules R2/R3
- orders the actions inside a recommendation package
- shown in the console with its countdown (fixture example: p_stuck 0.41, countdown 38 s)
- stored in the recommendation with model versions and a features hash; the ledger decision points to it, so it records **what the operator saw**

BUILD PHASE

- Finer bins near the contour; re-fit with S1/S5 surges as negatives
- **HistGradientBoosting** ensemble with isotonic calibration, kept only if it beats the grid on **both** Brier and AUC
- A false-critical check on surge runs before quoting a threshold
- Test set C, run exactly once

From the repo: data/models/forecaster_report.json (sets A/B, threshold sweep) · m3_detection/SPEC.md §6 · docs/DEVIATIONS.md (2026-10-25 M3-T01, M5 hero). Method follows Isaacs et al., HotOS 2025: learn vulnerable regions from a simulator. All simulation.

Bounded, right-sized actions that a human approves in one key. Unsafe ones can't be sent.

Models detect; **rules decide what is eligible**; humans decide what happens. Each action lists its **side effect** and **rollback condition**. Approvals *and rejections* are written to the ledger.

RULE TABLE R0–R7: EVALUATED IN ORDER, FIRST MATCH WINS

RULE	WHEN	RECOMMENDS
R0	DEGRADED_MONITORING	HOLD_MONITOR, ESCALATE_HUMAN only
R1	SWITCH_STATUS_OVERLOAD	DEFER_CHECKS (switch), light THROTTLE_RETRIES
R2	CRITICAL + PSP_SIDE(p)	SHED_EXPIRED, THROTTLE_RETRIES, PRIORITIZE_FRESH, SHIFT_TRAFFIC *if eligible*
R3	CRITICAL + BANK_SIDE(b)	THROTTLE_RETRIES(b), DEFER_CHECKS(b), SHED_EXPIRED, PRIORITIZE_FRESH; **SHIFT_TRAFFIC blocked** → DO_NOT_SHIFT; PACE_PAYOUTS if payouts > 10% of b
R4	WARNING only	DEFER_CHECKS (segment), THROTTLE_RETRIES at 0.2
R5	ROUTE_SPECIFIC(p,b)	THROTTLE_RETRIES(route), DEFER_CHECKS(b), route shift only if b is healthy elsewhere
R6	MULTI_FAULT / PLATFORM_WIDE	THROTTLE_RETRIES, DEFER_CHECKS, ESCALATE_HUMAN
R7	SURGE annotation	HOLD_MONITOR: "demand surge, queue draining, no action"

ACTION CATALOGUE, ORDERED LIGHTEST FIRST

THROTTLE_RETRIES: 10–20% of fresh rate, retry-after 2–8 s

DEFER_CHECKS: budget ×0.5, first check ×1.5

CHECK_BUDGET: per-PSP token bucket (no approval)

SHED_EXPIRED: drop work whose client already gave up

PRIORITIZE_FRESH: serve fresh before stale (LIFO idea)

PACE_PAYOUTS: ≤ 0.5 × bank's spare, ≥ 5/s

SHIFT_TRAFFIC: last resort, ≤ 25% steps, ≥ 10 s apart

ROLLBACK: proposed when the segment is clean for 6 ticks

HOLD_MONITOR · ESCALATE_HUMAN

Recommendations expire after 120 sim-s

ELIGIBILITY GATE FOR SHIFT_TRAFFIC (SI-2)

1. Verdict is **not** BANK_SIDE, SWITCH_OVERLOAD, MULTI_FAULT or PLATFORM_WIDE? — no → BLOCK
2. Every target PSP keeps **≥ 30% spare** after receiving its share? — no → BLOCK
3. No target has an open incident, or a WATCH/CRITICAL forecast? — no → BLOCK
4. Step ≤ 25% of source weight, ≥ 5% trickle stays, steps ≥ 10 s apart with jitter — else clamp

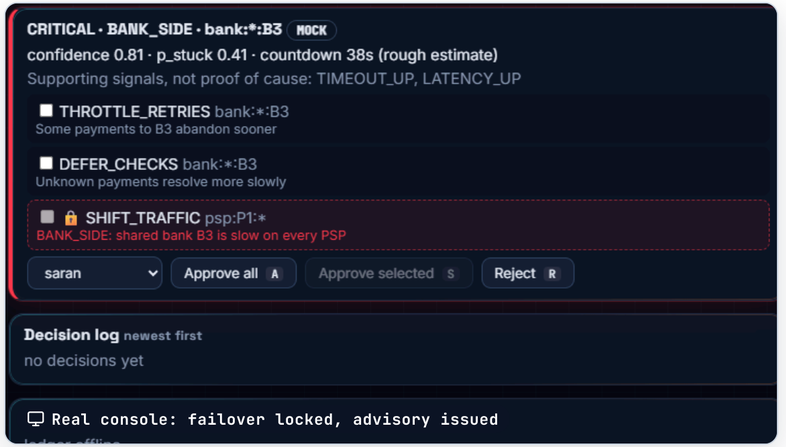
✓ **Eligible.** In-flight and unknown payments are **never moved** (M2 enforces).

Blocked actions appear with `blocked_by` naming the failed check. They are locked in the UI, **cannot be approved**, and turn into a signed `DO_NOT_SHIFT` advisory when the package is approved.

VERIFICATION GOVERNOR: THE LIVE CHECK BUDGET

```
spare = max(0, C_status × 0.85 - used)
rate_p = clamp(0.5 × spare × share_p,
r_min = 1, r_max = 0.3 × C × share_p)
burst_p = ⌈2 × rate_p⌉
```

Keeps the switch below its **slowdown knee** (85%). Defaults mirror OC-215: first check after 10 s (analogue of 90 s), 3 checks per 120 s horizon, after a 10 s ledger wait. Published only on > 10% change. **Tested:** spare 0 → r_min; more switch use → lower rates (C-7).



APPROVE → ACT → RECORD

1. `POST /v1/decisions` : APPROVED, PARTIAL or REJECTED; the operator must be listed in `operators.yaml`
2. Eligible actions are queued for the network as `act-NNNNN`
3. `decisionlog:RecordDecision` stores model versions and a **features sha256**
4. Advisories carry `evidenceHash = sha256(recommendation)`
5. Ledger down? Actions still apply; retried every 10 s, and the UI shows "recording on ledger..."

SAFETY INVARIANTS (IN CODE)

SI-1 never re-attempt an unknown payment · **SI-2** shift only if eligible · **SI-3** no approved action without a recorded decision · **SI-4** degraded → monitor only · **SI-5** budgets within [r_min, r_max] · **SI-6** versions, hash and reasons stored · **SI-8** every action has a rollback condition · **SI-9** no advisory without a decision · **SI-10** no amounts, accounts or VPAs on the ledger

From the repo: m4_control/SPEC.md §2–§9 (governor, rule table, eligibility, decisions, invariants) · m4_control/STATUS.md (C-tests; 24 tests pass). Screenshot: the console against a fixture recommendation, ledger mock. Same pattern as Citi Payment Outlier Detection: the model flags, the client approves before release.

It runs: five services, one operator console, eight demo stories, replayable frame-for-frame

Screens below are the **real M5 console**. In live mode, all five modules run together on one laptop with the ledger gateway in mock mode. The demo plays **recordings of simulation runs**, so every take is identical. The console labels them as recordings on screen.

THE 4-MINUTE DEMO VIDEO

1. Overview: what you're looking at
2. Festive surge stays quiet
3. Bank slowdown, localised
4. Human approval → ledger record
5. Unsafe failover, locked
6. April-2025 split replay
7. Arena: U0 vs U2 vs U4
8. About & claims

1080p, AI voiceover, burned-in subtitles plus an .srt file, one clip per feature. In `02_Demo_Video/`.

WATCH IT YOURSELF IN 2 MINUTES

```
cd 03_Source_Code/queueshield/m5_console
npm install && npm run dev
# no backend needed:
localhost:8000/?mode=replay
  &rec=s3_bank&speed=2&demo=1
```

Then open **Split** (8×, play) and **Arena**. Full live stack: Python 3.11 + Node 20, five services; see the README's quick start.

CONSOLE DETAILS

- WebGL scene (three.js) with a **heat-map fallback** and a reduced-motion mode
- Status chips for every service: live · replay · mock · offline
- Keyboard approve: **A** all · **S** selected · **R** reject
- **Claims lint** in tests: banned words fail the build
- Replay determinism tested (U-6)
- Smooth at 1920×1080 in Chrome (we saw ~80–114 fps)



Normal traffic. 3 PSPs, the switch and 5 banks on an illustrative India map. Particle colour = route health, check particles = switch load.



Traffic jumps and nothing fires. Latency is stable and queues drain: the surge gate labels it SURGE, not an incident.



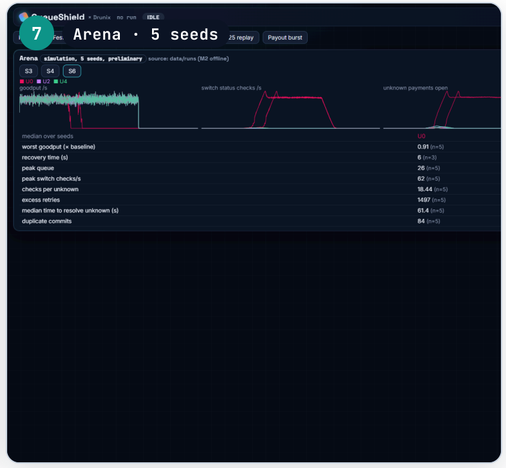
B1's column lights up for every PSP. Verdict BANK_SIDE. A package of bounded actions waits for approval, with reasons shown.



One key approves. The decision log shows each approval with its **tx id and block**, and advisories become active on the Drunix panel.



Same fault, same seed, one shared scale. No control peaks at **2,570 checks/s** with ≈4,370 unknowns open. QueueShield peaks at **38** with 0 SIM



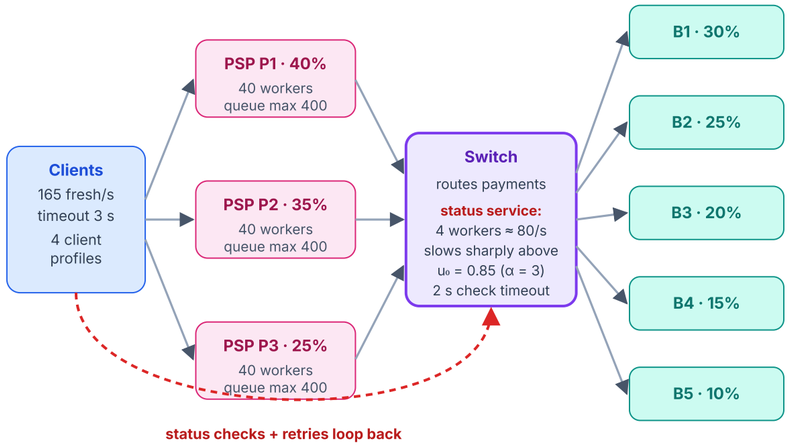
Paired comparison of U0, U2 and U4 on the same random tape: medians of every metric, labelled *simulation, preliminary*.

Screens: M5 console at 1920×1080 replaying recordings s1_festive, s3_bank, s6_u0/s6_u4 (seed 3) and the arena over data/runs. Tile 5 (failover locked) is on the previous slide. Map positions are illustrative: the simulator has no geography.

How we measure: a reactive UPI-like network, replayed on identical seeds under five policies

We can't break real UPI, so we built a **reactive** simulator: queues, timeouts, retries, zombie work, shared banks and a shared status service. **Common random numbers** mean every universe sees the same payments and faults, so **differences come from policy only**.

M2 NETWORK MODEL (CONFIG/SIM_DEFAULT.YAML)



Each payment has a **remitter leg and a beneficiary leg** (mean 150 ms, CV 0.6), with business and technical decline rates, zombie work, reversals after 20 s, and idempotency keys. Ground truth stays inside M2.

REAL CONSTANTS AND THEIR ANALOGUES (TIME COMPRESSED)

CONSTANT	REAL	SIM	RATIO
Pay timeout (OC-214)	15 s	3 s	5:1
Status response cap	10 s	2 s	5:1
First check (OC-215)	90 s	10 s	9:1
Max checks	3 in 2 h	3 in 120 s	60:1
TAT, debit w/o credit	T+1 day	120 s	≈720:1

NINE SCENARIOS (PARAMETERS DRAWN PER RUN)

ID	STORY	FAULT
S0	Normal	baseline, false-alert rate
S1	Festive surge	demand ×1.4–1.8, flash ×2.5 for 20 s: **must stay quiet**
S2	PSP slowdown	one PSP's service ×1.6–3.0
S3	Bank slowdown	one bank's legs ×4–8 on **every** PSP
S4	Retry storm	a PSP loses 50–70% of workers for 40–90 s
S5	Surge + slowdown	S1 and S2 together
S6	April-2025 replay	bank B1 black-holes 10–30% of credits for 15–60 s
S7	Silent drift	one route ramps ×1 → ×2.2 over 120 s
S8	Bulk payout	2,000–4,000-payment batch into a slow bank

FIVE UNIVERSES ON THE SAME TAPE

U	CLIENTS	BEHAVIOUR
U0	naive	no control; checks every 2 s; users retry (pre-April 2025)
U1	naive	textbook failover on success rate
U2	npci_rule	**OC-215 timers** on every PSP: the strong baseline
U3	governed	QueueShield without the ledger
U4	ledger_first	QueueShield + Drunix push + advisories (30% naive mix)

U3 and U4 keep **30% naive clients** on purpose: real networks never upgrade every app at once.

PROTOCOL: WRITTEN BEFORE THE RUNS

- **Common random numbers:** named PCG64 streams, so the same tape replays in every universe (tested, S-3)
- **Disjoint configuration sets:** A train (1–999), B validation (1000–1999), C test (2000–2999), with C run once after a tagged freeze
- **Paired by seed:** medians, IQR and 95% bootstrap CIs (10,000 resamples), plus a Wilcoxon sanity check
- **Lockstep batch mode:** M2 → M3 → M4 every 5 s tick; a simulated operator approves after 20 s
- Runs of ≈10 min at 165 payments/s (≈97,000 payments)

PRE-REGISTERED HYPOTHESES: STATUS TODAY

ID	CLAIM WE TEST	STATUS
P1	D3 detects sooner than D0–D2 at ≤ 1 false alert/h	◐ D3 before D0 on S3; tuning open
P2	surges stay quiet	◐ S1 recording quiet; top of S1 range is not benign
P3	U4 recovers better than U0/U2 on S4	✓ **prelim: 0/5 vs 3/5, 2/5 stuck**
P4	localisation prevents harmful failover	◐ U1 failover collapse seen; S3 pair not run
P5	governed checks cut the switch peak	✓ **S6: 62 → 36/s median**
P6	forecaster gives lead time	◐ 9/14 before collapse, 0 false
P7	ledger removes residual checks, 0 duplicates	✗ **not yet: 7.2 vs 3.0; 21 dup.**
P8	Drunix carries the exception path	benchmark not run

Threats to validity: circularity (we wrote both the simulator and the detectors), a small topology (3 PSPs, 5 banks), a simulated operator, an assumed switch coupling, and only 5 seeds at pitch stage. Mitigations: held-out sets, the U2 baseline, and negative results reported.

From the repo: m2_simulator/config/sim_default.yaml and scenarios.yaml · docs/00_PRODUCT.md §6–7 · docs/90_PLAN_EVAL_RISKS.md §6 · m2_simulator/STATUS.md (29 tests; arena 2026-10-25). Simulation only; calibrated toward public NPCI statistics, not validated on real bank traffic.

April-2025 replay: no control locked into a storm in 2 of 5 runs. QueueShield in 0 of 5.

A beneficiary bank black-holes some of its credits. Without control, apps poll the switch until it saturates. **Both** the NPCI rule and QueueShield keep every seed out of the storm. QueueShield escalates **29% fewer** payments to manual handling, but still uses **more checks** than the NPCI rule.

MEDIANS OVER SEEDS 1-5 SIMULATION

METRIC	U0 NONE	U2 NPCI	U4 QS
Runs locked into a storm	**2 / 5**	**0 / 5**	**0 / 5**
Peak switch checks / s	62	**22**	36
Switch checks per unknown	18.4	**3.0**	**7.2**
Switch checks, total	3,602	**705**	1,689
Escalated to manual handling	0	235	**166**
Duplicate commits	84	**0**	21
Unknown payments	234	235	235
Unique successful payments	97,090	96,970	97,000
Median time to resolve (s)	61.4	50.1	50.2
p95 time to resolve (s)	61.7	50.2	61.6
Worst goodput (× baseline)	0.914	0.924	0.925
Peak PSP queue	26	0	2

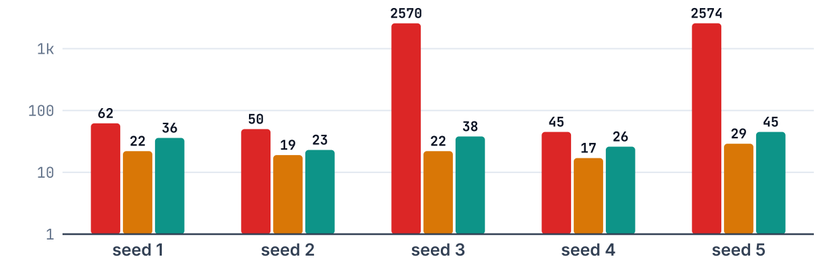
Medians hide the storms: in the 2 locked U0 runs, the switch peaked at **2,570–2,574 checks/s** and only **40–50k** payments completed, against ~97k normally. U0 escalates nothing because it never stops asking. Recovery times are omitted: they are measured only on recovered runs, so they aren't comparable.

SWITCH CHECKS PER UNKNOWN, EVERY SEED

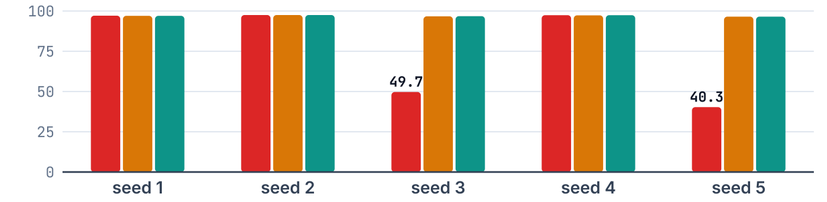
U	S1	S2	S3	S4	S5
U0	15.4	18.4	**29.6**	16.6	**29.7**
U2	3.0	3.0	3.0	3.0	3.0
U4	7.2	7.3	7.1	6.4	7.3

U2 is exactly 3.0 because the rule caps every payment at 3 checks. U4 is stable across seeds, including the storm seeds.

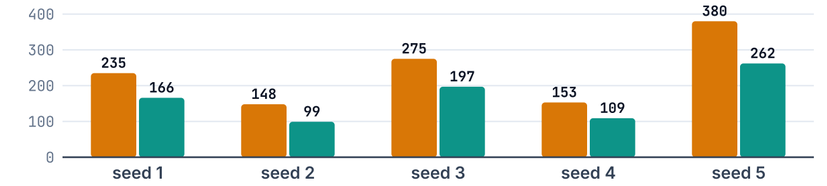
PEAK SWITCH CHECKS PER SECOND, PER SEED (LOG SCALE)



PAYMENTS COMPLETED PER RUN (THOUSANDS)



ESCALATED TO MANUAL HANDLING, U2 VS U4, PER SEED



U4 escalates fewer in every seed (−28% to −33%), because the ledger resolves those unknowns from bank attestations.



2,570/s
peak checks, U0, seed 3, with ≈4,370 unknowns open

38/s
peak checks, U4, same seed and fault, 0 unknowns open at the end

% HEADLINE DELTAS SIM

Checks per unknown, U4 vs U0	−61%
Escalations, U4 vs NPCI rule	−29%
Duplicate commits, U4 vs U0	−75%
Checks per unknown, U4 vs NPCI rule	+140%
Duplicates, U4 vs NPCI rule	21 vs 0

Why U4 loses to U2 on checks: U4 keeps **30% naive clients** by design, and they poll like U0. A per-profile breakdown is the first build-phase analysis. Those naive clients also cause U4's duplicates.

Data: data/runs/r-S6-{1..5}-U{0,2,4}/result.json, packaged as 06_Results/S6_medians_simulation.csv and runs_metrics_simulation.csv. Lockstep batch with M3+M4 and a simulated operator (20 s delay). Synthetic data; 5 seeds is a pitch-stage sample (build phase: ≥ 20 configurations with bootstrap CIs).

Retry storm: no control locked in on 3 of 5 seeds, the NPCI rule on 2 of 5, QueueShield on 0

A PSP loses 50–70% of its workers for 40–90 s. Timeouts trigger retries and status checks that outlive the fault. This is where the NPCI timers alone **were not enough** in our simulator, and where QueueShield's network-level response shows most clearly.

EVERY RUN, EVERY UNIVERSE SIMULATION

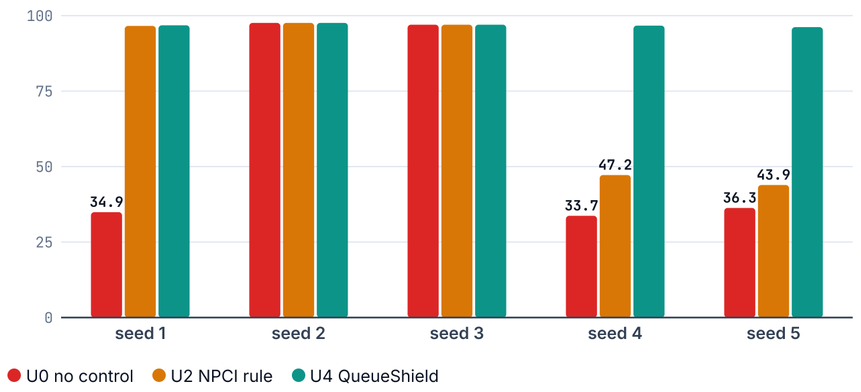
	seed 1	seed 2	seed 3	seed 4	seed 5
U0 none	STUCK 34.9k ok	no storm 97.6k ok	no storm 97.0k ok	STUCK 33.7k ok	STUCK 36.3k ok
U2 NPCI rule	recovered 96.6k ok	no storm 97.6k ok	no storm 97.0k ok	STUCK 47.2k ok	STUCK 43.9k ok
U4 QueueShield	recovered 96.8k ok	no storm 97.6k ok	no storm 97.0k ok	recovered 96.7k ok	recovered 96.2k ok

"ok" = unique successful payments in the run (≈97k when healthy). Seeds 2 and 3 drew short faults that recovered on their own in every universe.

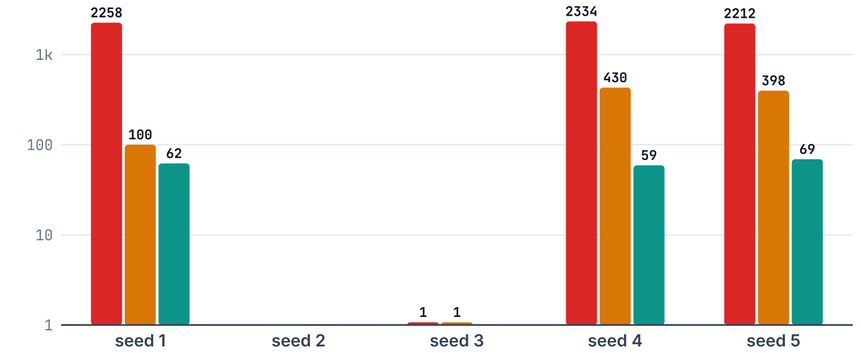
MEDIANS OVER SEEDS 1-5 SIMULATION

METRIC	U0	U2	U4
Runs locked into a storm	**3 / 5**	**2 / 5**	**0 / 5**
Unique successful payments	36,302	96,582	**96,843**
Peak switch checks / s	2,212	100	**59**
Switch checks per unknown	29.6	2.1	**0.65**
Unknown payments	26,739	2,326	**1,821**
Duplicate commits	1,900	**0**	30

PAYMENTS COMPLETED PER RUN (THOUSANDS)



PEAK SWITCH CHECKS PER SECOND (LOG SCALE)



Seed 2 (all universes) and seed 3 (U4) had no switch checks at all, so no bar is drawn.

WHAT THIS SHOWS

In the two seeds where NPCI's timers alone could not stop the storm, QueueShield recovered, completing ≈2× as many payments (96.7k vs 47.2k; 96.2k vs 43.9k) SIM

% HEADLINE DELTAS, MEDIANS SIM

Peak switch checks, U4 vs U2	−41%
Checks per unknown, U4 vs U2	−69%
Checks per unknown, U4 vs U0	−98%
Unknowns, U4 vs U2	−22%
Duplicates, U4 vs U2	30 vs 0

READ WITH CARE

- **5 seeds, preliminary.** No confidence intervals yet.
- Not yet separated: how much comes from the **ledger push** and how much from **control actions**. The U3 run (QueueShield without the ledger) is build-phase work.
- In our **live S4 recording**, detection raised **no incident** at current calibration. We have not yet broken out whether detection fired in each arena run.
- Duplicates in U4 come from the 30% naive clients. The fix is an idempotency guard before any re-attempt.

Also measured in S4: textbook failover (U1) made things worse. Every run where it moved P1's traffic collapsed the whole network (goodput afterwards 0/s, against 18/s with no control).

Data: data/runs/r-S4-{1..5}-U{0,2,4}/result.json (06_Results/runs_metrics_simulation.csv) · m2_simulator/STATUS.md preliminary arena (2026-10-25) and the U1 finding. "Stuck" = goodput below 50% of baseline after the trigger. Deltas are on medians. Synthetic data, our simulator.

Where we win, where we don't yet, and what is real versus simulated

The NPCI rule is a **strong** baseline and we compare against it, not just against doing nothing. Every open item has a **named fix** scheduled for the 10–22 Nov build phase.

WORKING, WITH EVIDENCE

- **0 storm lock-ins** in U4 across S4 and S6 (10 runs)
  vs U0 5/10 and U2 2/10 · arena, 2026-10-25 SIM
- **≈2× the payments completed** in the 2 S4 seeds where the NPCI rule locked in
  96.7k vs 47.2k; 96.2k vs 43.9k SIM
- **−29% escalations** to manual handling vs the NPCI rule, in every S6 seed
  166 vs 235 median; per seed −28% to −33% SIM
- **Right bank named** 15 s after the fault (S3)
  BANK_SIDE · B1; localiser ≥ 95% on 6 verdicts × 1,000 trials (test D-6)
- **Festive surge stays quiet** (S1 recording)
  zero non-monitoring ticks in the recorded run
- **Unsafe failover locked**; DO_NOT_SHIFT advisory issued
  eligibility checks in code (SI-2); console verified on a fixture
- **Decisions and advisories on the ledger** (mock), with tx ids
  70 golden vectors; 100 concurrent attestation pairs; M2 bridge test
- **106 automated tests** pass across five modules
  M1 7 · M2 29 · M3 25 · M4 24 · M5 21; tsc clean; npm audit 0

OPEN, WITH THE PLANNED FIX

- **More checks than the NPCI rule in S6** (7.2 vs 3.0 per unknown)
  Fix: per-profile breakdown (30% naive clients); tighter DEFER_CHECKS budgets; enforce E_STATUS_STOP
- **Duplicate commits** in U4: 21 (S6) and 30 (S4) vs 0 under the NPCI rule
  Fix: idempotency guard: no re-attempt until the ledger says FAILED_NO_DEBIT
- **Live S4 recording raised no incident** at current calibration
  Fix: matched-budget tuning (D-11) and S4 recalibration
- **Forecaster grid too coarse to draw**; only 9/14 warned before collapse at 0.6
  Fix: finer bins, surge negatives, and a gradient-boosting comparison on set B
- **Live Drunix network not yet run**; Go chaincode not yet compiled
  Fix: bring-up on Linux/WSL2 (11 containers), run L2–L8 live, benchmark P8
- **Recommendations aimed one level off** in the S3 recording (psp:P3 / route:P1:B3, not bank B1)
  Fix: require the localiser primary to warn for 1 tick before opening; merge route incidents into the bank
- **Only 5 seeds**; no confidence intervals; U1/U3 ablations missing
  Fix: ≥ 20 configurations per scenario, bootstrap CIs, U3 to separate ledger from control
- **S1 is not benign at the top of its range** (ρ ≈ 1.0 locks in)
  Fix: cap the surge multiplier so the busiest PSP stays ≤ ~0.9 (dataset change)

REAL

All code: simulator, detectors, localiser, forecaster, rules, governor, console; Go chaincode and TS gateway; 17 JSON Schemas; 106 passing tests; the integration smoke across all five services.

SIMULATED

All payment traffic, PSPs, banks and the switch. All data is synthetic. Time is compressed (labelled analogue). The operator in batch runs is simulated (approves after 20 s).

MOCK

The ledger in the demo: the gateway's **mock mode**, with the same REST/SSE routes, schemas and events, and badged "mock" in the console. It never fakes ledger numbers.

WHAT WE NEVER CLAIM

`"prevents outages"` `"predicts outages"` `"production-ready"` `"NPCI-approved"` `"Citi integration"` `"scales to UPI"` `"guarantee"`

The console's **claims lint** fails the test suite if any UI text uses them.

THREATS TO VALIDITY

- **Circularity:** we wrote both the simulator and the detectors. Mitigations: held-out sets, the U2 baseline, ablations.
- **Small topology:** 3 PSPs and 5 banks; the switch coupling is an assumption.
- **Simulated operator** with a fixed 20 s delay.
- **Laptop-scale ledger**: throughput not yet measured.

Sources: data/runs (arena 2026-10-25) · module STATUS.md files · docs/DEVIATIONS.md · docs/00_PRODUCT.md §8 (claims policy). Every "Open" item is a task card in the module ROADMAP.md files.

Who gains, by how much in our simulator, and how this could reach a real network

Judged on **Innovation × Scale × Impact**. Every number below is from our simulator SIMULATION. The adoption path is a **proposal**: each step needs NPCI and member-bank agreement.

Innovation

- Treats unknown payments as a **network load** problem, not one payment at a time
- A **lock-in forecaster** learned from a reactive simulator (the HotOS 2025 method)
- **Signed advisories and decisions** on the same ledger as the payment truth
- A **check budget** sized from spare switch capacity, live

Scale

- Only exceptions touch the ledger: **≈0.24%** of payments in our runs SIM
- At UPI volume that is ≈22–46 unknowns/s on average (our arithmetic), on **stateless, horizontally scalable** Drunix peers
- Detection is O(segments) per tick; the forecaster is two lookups
- Sits **beside** the rail: it can't slow a payment

Impact

- **0 of 10** storm lock-ins (S4 + S6), vs 5/10 with no control and 2/10 with the NPCI rule SIM
- **≈2×** the payments completed where the NPCI rule locked in (S4) SIM
- **−29%** manual escalations vs the NPCI rule (S6) SIM
- A signed, auditable trail for the regulator

VALUE BY STAKEHOLDER

WHO	WHAT CHANGES FOR THEM	MEASURED IN OUR SIMULATOR	COST TO THEM
Payers	A final status, pushed; no blind second payment; TAT enforced on-ledger	duplicate commits −75% vs no control (S6); 0 vs NPCI rule still open	none
PSPs	Answers by subscription; a live check budget; alerts that name the segment	checks per unknown −61% vs no control (S6); −69% vs NPCI rule (S4)	subscribe to SSE; honour the budget
Banks	Attest once, signed by their own org, instead of answering repeated queries	peak switch checks 59/s vs 100/s under the NPCI rule (S4)	run a peer; sign attestations
NPCI network ops	Which segment, how soon it locks in, bounded actions approved in one key	bank named in 15 s (S3); 0/10 lock-ins	an operator approves
Regulator	Provable compliance: TAT breaches and decisions are recorded by the chaincode, not self-reported	TAT breach emitted exactly once (logic tested)	a read-only node (planned)
Sponsor bank Citi-like role	Early warning on beneficiary banks; payouts paced instead of piling into a sick bank	scenario S8 built; results pending	respect PACE_PAYOUTS

A PROPOSED PATH TO A REAL NETWORK (EACH STEP GATED ON EVIDENCE)

NOW
Simulator + mock ledger
5 modules, 106 tests, paired-seed arena

10-22 NOV
Live Drunix
measured latency and tx/s, ≥ 20 configs, fixes from the scorecard

NEXT
Sandbox with NPCI APIs
if offered to selected teams: replay real-shaped traffic

THEN
Shadow mode
read-only on events; recommendations logged, never executed

LATER
Advisory pilot
a few PSPs and banks attest and subscribe; humans approve everything

WHAT A PILOT WOULD NEED TO DEPLOY

Each bank
a Drunix Lite Peer + its org identity; it signs attestations for its own legs

Each PSP
opens unknowns; subscribes to SSE; honours the published check budget

Network ops
M3 + M4 on the switch's event stream; one operator console (M5)

Regulator
optional read-only node: TAT breaches, advisories, decisions

WHY IT FITS NPCI'S DIRECTION

It **follows the principle** of OC-215 (check less, check later, stop on certain errors) and of the peak-hour API limits, but makes them **load-aware, network-wide and provable**, on NPCI's own ledger stack. No change to how a payment is routed or settled.

- ✓ OC-215: first check delay, max checks, stop codes → **default check budget**
- ✓ OC-214: faster timeouts → **more unknowns, sooner** , so push matters more
- ✓ Peak-hour API limits → **load-aware budgets** instead of fixed caps
- ✓ RBI TAT → **clock on the ledger** , breach recorded once
- ✓ Drunix → NPCI's **own** permissioned stack

Data: arena runs S4/S6 seeds 1–5 (data/runs), recording s3_bank, m1 golden vectors. Arithmetic: 802 M/day × 0.24–0.5% ÷ 86,400 s. The adoption path is our proposal, not an agreement with NPCI, Citi or any bank. All outcome numbers: simulation, synthetic data.

Built and tested today: ≈11.7k lines across five modules, 106 passing tests, every status evidenced

Each module has an owner, a spec, a status file where ✅ **means verified by a test that was run**, and a roadmap of task cards with acceptance checks. Deviations from the spec are logged, never hidden.

MODULE BY MODULE

MODULE	STACK	LOC	TESTS	✅ VERIFIED	🟡 BUILT, NOT VERIFIED · ⬜ NEXT
M1 Ledger :8004	Go chaincode, TypeScript gateway	1.5k	7	mock gateway: every route, same codes; 70 golden vectors; SSE ring + Last-Event-ID; keyed mutex (100 concurrent pairs); used by M4 live	🟡 Go state machine (no Go here) · 🟡 TAT sweeper loop · ⬜ live Drunix bring-up
M2 Simulator :8001	Python 3.11, numpy, FastAPI	3.2k	29	engine, legs, declines, zombie work; CRN tape; 4 client profiles; ledger model; ledger bridge to the real M1 gateway; arena jobs; U1 + all actions + rollback (test_paths)	🟡 S8 results · ⬜ E_STATUS_STOP for npci_rule · ⬜ S1 range cap
M3 Detection :8002	Python 3.11, numpy, FastAPI	2.6k	25	ingest/dedupe/watermark; 23 features; EWMA baselines; D3 rules; localiser ≥ 95%; grid fit + training split; model timing guard; lifecycle	🟡 D0–D2 thresholds · ⬜ matched-budget tuning · ⬜ IsolationForest
M4 Control :8003	Python 3.11, numpy, FastAPI	1.6k	24	rules R0–R7 (one test per row); eligibility; governor (C-7); decisions → actions → ledger; simulated operator; batch endpoints	🟡 operator-delay sweep written, not run · ⬜ maker-checker for shifts
M5 Console :8000	React, Vite, TypeScript, three.js	2.8k	21	3D scene + mapping tests; replay determinism (U-6); claims lint; split replay; arena; approve flow; tsc clean; npm audit 0	⬜ regulator read-only view · ⬜ live Drunix panel
Cross-module	all five	—	smoke	integration smoke: incidents raised → recommendation approved → decision on the ledger mock → actions applied → incidents closed	⬜ the same on live Drunix

LOC: Python / TypeScript / Go source per module (`git ls-files`) Tests: last runs, 2026-10-25 · total 106

REPOSITORY LAYOUT

```
queueshield/
├ contracts/ schemas (17) · fixtures · PROPOSALS
├ docs/ product · system · plan · research · deviations
├ m1_ledger/ chaincode/ gateway/ network/ testdata/
├ m2_simulator/ qsim/ config/ tests/
├ m3_detection/ qdet/ scripts/ tests/
├ m4_control/ qctl/ config/ tests/
├ m5_console/ src/ tests/ scripts/
└ data/ recordings · runs · models · videos
each m*/: README · STATUS · ROADMAP · SPEC · CLAUDE
```

ENGINEERING PRACTICES

- **Contract-first:** no module imports another; invalid messages are dropped and counted
- **Golden vectors** shared by the Go and TypeScript state machines
- **Deterministic:** common random numbers (test S-3) and replay determinism (test U-6)
- **Claims lint** on all UI text; a "simulation" label on every outcome number
- **Write auth** via `X-QS-Key` ; no secrets in code; no PII on the ledger
- **Dependencies pinned** (e.g. three@0.186.1), audited, logged in DEVIATIONS

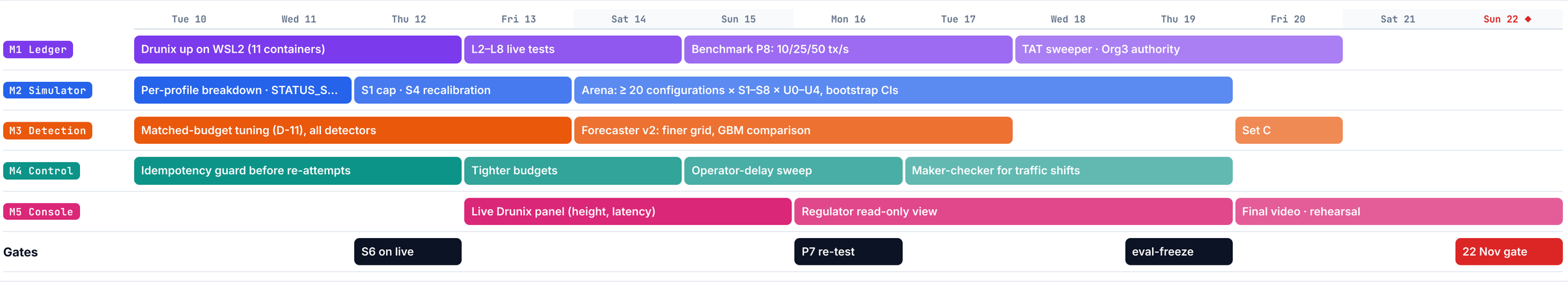
WHAT'S IN THE SUBMISSION

01 this deck (PDF + PPTX)
02 4-min narrated demo + 8 clips
03 full source + recordings
04 docs and module specs
05 1080p screenshots
06 per-run metrics (CSV)

From the repo: m1–m5 STATUS.md (legend: ✅ verified by a test that was run · 🟡 built, not fully verified · ⬜ not started) · m5_console/scripts/integration_smoke.py · docs/DEVIATIONS.md.

The build-phase plan: go live on Drunix, beat the NPCI rule on checks, close the duplicate gap

Every bar is a task card in a module ROADMAP.md with an acceptance check. Scorecard fixes come first. The 22 Nov gate: **every pre-registered hypothesis reported, held or not**, and a README that reproduces the demo in under 30 minutes.



RISK REGISTER

RISK	LIKELIHOOD · IMPACT	EARLY SIGNAL	FALLBACK (ALREADY PREPARED)
Drunix won't start	high · high	Day-1 `up` fails	WSL2; known fixes (ccenv 1.0, YugabyteDB wait); the mock keeps every module moving
No chaincode events from the Committing Peer	med · med	test L3 fails	subscribe via the Lite Peer, or low-rate GetStatus polling inside the gateway, documented
Fixes don't beat the NPCI rule on checks	med · high	P7 re-test	report it; lead with lock-in, escalations and S4, where we already win
Metastability not robust after recalibration	med · med	calibration grid	rename to a "congestion forecaster"; same method (spec §6.6)
Overclaiming	med · high	banned words	claims lint in tests + a second reviewer on every number
Live demo fails at the finale	med · high	rehearsal	replay mode (deterministic) + backup video
Originality questions	med · v.high	rules text	disclosed up front; reused parts listed in REUSE_MAP
A team member is unavailable	med · med	missed stand-up	self-contained modules; the backup reviewer takes over from the module docs

SUCCESS CRITERIA FOR 22 NOV

- [ ] S6 end to end on a **live** Drunix network, with measured submit/event latency
- [ ] U4 checks per unknown **≤ U2** on S6, with a per-profile breakdown
- [ ] **0 duplicate commits** in U3/U4 after the idempotency guard
- [ ] S4 detected live; forecaster lead time reported on set C
- [ ] ≥ 20 configurations per scenario, with medians, IQR and bootstrap CIs
- [ ] Regulator view; README demo in < 30 min from a clean machine

WHAT WOULD HELP US MOST

Access to **NPCI sandbox APIs or real-shaped traffic statistics**, so we can calibrate the simulator beyond public data. A **Drunix reference deployment**, to measure throughput on the real stack.

FINALE, 4 DEC: WHAT WE WILL SHOW

- A **live** Drunix network running the `queueshield` chaincode, with measured latency on screen
- S6 and S4 end to end, with an operator approving live and the decision tx on the ledger
- The full arena with confidence intervals, including every hypothesis that did **not** hold
- A regulator view of TAT breaches and decisions

From the repo: docs/PROJECT_ROADMAP.md · docs/90_PLAN_EVAL_RISKS.md §5–§7 · module ROADMAP.md task cards. Dates: hackathon timeline as published by participating teams (Project Axis README); the official dashboard is authoritative.

Machines recommend, humans decide, the ledger remembers.

QueueShield × Drunix finds the sick bank before a status-check storm locks in, budgets the checks, and resolves unknown payments by **push from a shared ledger**, not by polling the switch.

THREE THINGS TO REMEMBER

1. A timeout is unknown, not failure, and asking about it is load.
2. The bank that knows attests once; everyone else subscribes.
3. Measured against NPCI's own rule, with our losses on the slide.

0 / 10
storm lock-ins with QueueShield across S4 + S6 (vs 5/10 no control, 2/10 NPCI rule) SIM

≈0.24%
of payments touch the ledger: exceptions only SIM

15 s
from bank fault to "BANK_SIDE · B1" in the S3 replay SIM

106
automated tests passing across 5 modules

WHERE TO LOOK NEXT

02_Demo_Video/ the 4-minute narrated demo · **03_Source_Code/** run the replay with no backend · **06_Results/** every run's metrics as CSV

Team: Achilles

Team lead: Sanjay R · 24z241@gmail.com

All outcome numbers come from our simulator, with synthetic data. The ledger demo runs on the gateway's mock mode. This follows the principle of NPCI's April 2025 circulars in a scaled analogue. No connection to NPCI or Citi systems.

JUDGE Q&A: SHORT, HONEST ANSWERS

Why a blockchain and not a database?
The truth is split across organisations. Each fact is signed by the org that knows it, the endorsement policy enforces who may say what, and compliance with TAT and the rules becomes provable rather than self-reported.

Isn't this just Pulse or Tally?
They make one payment's truth honest, and we include that pattern. We add the network level: localisation, lock-in forecast, check budgets, signed decisions, and paired-seed evaluation against the NPCI rule.

Why does QueueShield use more checks than the NPCI rule?
U4 keeps 30% naive clients by design, and they poll. The build-phase target is U4 ≤ U2, with a per-profile breakdown. We show the loss rather than hide it.

Why a human in the loop?
These are financial and routing actions, and a timeout is not a confirmed failure. The operator-delay sweep (5–120 s) will show how fast approval needs to be.

What does a sponsor bank like Citi get?
Early warning on beneficiary banks, load-aware payout pacing (S8), and the same push-status model Citi Token Services uses. This is a role in our simulator, not an integration.

What if the ledger is down?
Actions still apply, ledger writes retry every 10 s, and the console shows "ledger offline". We never fake ledger events.

Can it handle UPI scale?
Only exceptions go on-chain: ≈22–46 unknowns/s on average at UPI volume by our arithmetic. We haven't measured Drunix throughput yet; that is benchmark P8. Lite Peers and Validation Services scale out.

Do banks have to change anything?
They attest the legs of unknown payments they already know about. That is less work than answering the same status query again and again.

Is the simulator realistic?
It models queues, timeouts, retries, zombie work, shared banks and a shared status service, and compares policies on identical random draws. It is not real bank traffic, and we say so on every number.

Why no LLM?
Determinism, latency and auditability. An LLM could later explain an incident in plain words, outside the decision path.

Is any of this original to this hackathon?
The detection core began in our IBM Z Datathon entry and is listed in REUSE_MAP. The Drunix ledger, UPI model, localiser, forecaster training and this build are new.

What would you do with NPCI access?
Calibrate against real-shaped traffic statistics, run shadow mode (read-only, recommendations logged and never executed), and measure Drunix on NPCI's reference deployment.

Sources: docs/90_PLAN_EVAL_RISKS.md §8 (judge Q&A) · results slides 15–16 · docs/REUSE_MAP.md. Thank you to NPCI and Citi for the DRUNIX Hackathon 2026.